2026

Apache Spark

Name: Muhammad Fahad Ullah

Roll No: 231980071

Instructor: Sir Zuhaib Hussain Butt

Course: Big Data Analytics

Part 1: Environment Setup

- Python Version
- Spark Version
- Operating System
- Java Version

```
Python Version: 3.12.13 (main, Mar  4 2026, 09:23:07) [GCC 11.4.0]
```

```
!java -version
```

```
openjdk version "17.0.19" 2026-04-21
OpenJDK Runtime Environment (build 17.0.19+10-1-22.04.2-Ubuntu)
OpenJDK 64-Bit Server VM (build 17.0.19+10-1-22.04.2-Ubuntu, mixed mode, sharing)
```

```
PySpark Version: 4.0.3
```

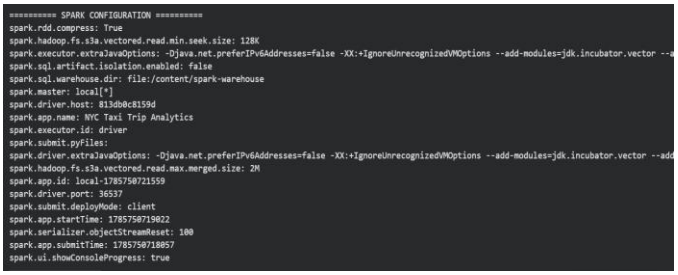
```
Spark Session Created Successfully!
Spark Version: 4.0.3
Master: local[*]
```

```
import platform
import sys

print("Python Version:", sys.version)
print("Operating System:", platform.system())
print("Spark Version:", spark_session.version)
```

```
Python Version: 3.12.13 (main, Mar  4 2026, 09:23:07) [GCC 11.4.0]
Operating System: Linux
Spark Version: 4.0.3
```

- Spark configuration



Part 2– Load Dataset

```
Dataset Loaded Successfully!
```

- Schema

```
root
 |-- VendorID: integer (nullable = true)
 |-- tpep_pickup_datetime: timestamp_ntz (nullable = true)
 |-- tpep_dropoff_datetime: timestamp_ntz (nullable = true
 |-- passenger_count: long (nullable = true)
 |-- trip_distance: double (nullable = true)
 |-- RatecodeID: long (nullable = true)
 |-- store_and_fwd_flag: string (nullable = true)
 |-- PULocationID: integer (nullable = true)
 |-- DOLocationID: integer (nullable = true)
 |-- payment_type: long (nullable = true)
 |-- fare_amount: double (nullable = true)
 |-- extra: double (nullable = true)
 |-- mta_tax: double (nullable = true)
 |-- tip_amount: double (nullable = true)
 |-- tolls_amount: double (nullable = true)
 |-- improvement_surcharge: double (nullable = true)
 |-- total_amount: double (nullable = true)
 |-- congestion_surcharge: double (nullable = true)
 |-- Airport_fee: double (nullable = true)
```

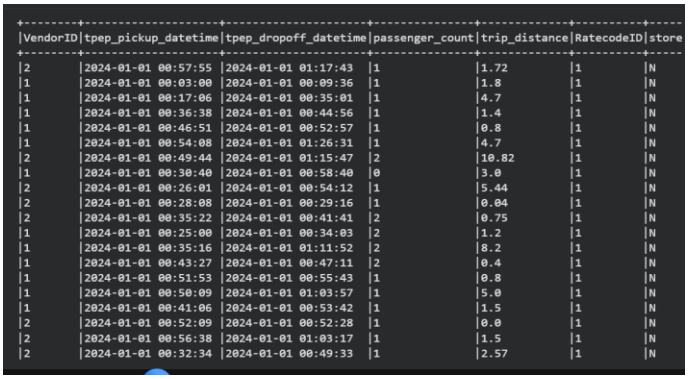
- Number of columns

```
Number of Columns: 19
```

- Number of records

```
Number of Records: 2964624
```

- First 20 rows



VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store
2	2024-01-01 00:57:55	2024-01-01 01:17:43	1	1.72	1	N
1	2024-01-01 00:03:00	2024-01-01 00:09:36	1	1.8	1	N
1	2024-01-01 00:17:06	2024-01-01 00:35:01	1	4.7	1	N
1	2024-01-01 00:36:38	2024-01-01 00:44:56	1	1.4	1	N
1	2024-01-01 00:46:51	2024-01-01 00:52:57	1	0.8	1	N
1	2024-01-01 00:54:08	2024-01-01 01:26:31	1	4.7	1	N
2	2024-01-01 00:49:44	2024-01-01 01:15:47	2	10.82	1	N
1	2024-01-01 00:30:40	2024-01-01 00:58:40	0	3.0	1	N
2	2024-01-01 00:26:01	2024-01-01 00:54:12	1	5.44	1	N
2	2024-01-01 00:28:08	2024-01-01 00:29:16	1	0.04	1	N
2	2024-01-01 00:35:22	2024-01-01 00:41:41	2	0.75	1	N
1	2024-01-01 00:25:00	2024-01-01 00:34:03	2	1.2	1	N
1	2024-01-01 00:35:16	2024-01-01 01:11:52	2	8.2	1	N
1	2024-01-01 00:43:27	2024-01-01 00:47:11	2	0.4	1	N
1	2024-01-01 00:51:53	2024-01-01 00:55:43	1	0.8	1	N
1	2024-01-01 00:50:09	2024-01-01 01:03:57	1	5.0	1	N
1	2024-01-01 00:41:06	2024-01-01 00:53:42	1	1.5	1	N
2	2024-01-01 00:52:09	2024-01-01 00:52:28	1	0.0	1	N
2	2024-01-01 00:56:38	2024-01-01 01:03:17	1	1.5	1	N
2	2024-01-01 00:32:34	2024-01-01 00:49:33	1	2.57	1	N

Part 3– Exploratory Data Analysis

- Total trips

```
Total Trips: 2964624
```

- Earliest trip date

Earliest_Trip_Date
2002-12-31 22:59:39

- Latest trip date

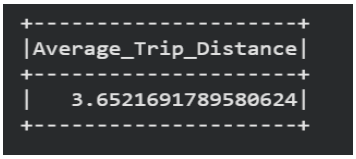
Latest_Trip_Date
2024-02-01 00:01:15

- Unique vendors

VendorID
1
2
6

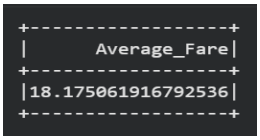
```
Number of Unique Vendors: 3
```

- Average trip distance



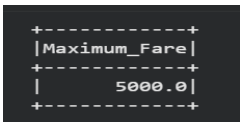
```
+---------------------+
|Average_Trip_Distance|
+---------------------+
|   3.6521691789580624|
+---------------------+
```

- Average fare



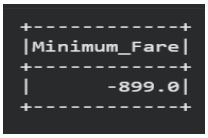
```
+------------------+
|      Average_Fare|
+------------------+
|18.175061916792536|
+------------------+
```

- Maximum fare



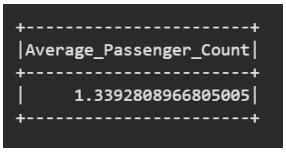
```
 +-------------+
 |Maximum_Fare|
 +-------------+
 |      5000.0|
 +-------------+
```

- Minimum fare



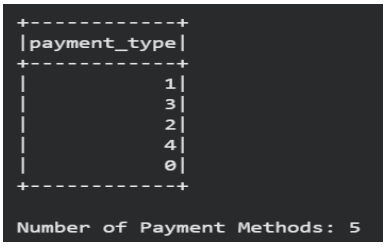
```
+-------------+
|Minimum_Fare|
+-------------+
|       -899.0|
+-------------+
```

- Average passenger count



```
+----------------------+
|Average_Passenger_Count|
+----------------------+
|     1.3392808966805005|
+----------------------+
```

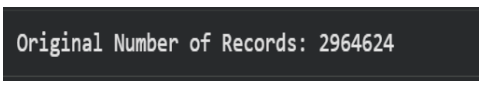
- Number of payment method



```
+-------------+
|payment_type|
+-------------+
|            1|
|            3|
|            2|
|            4|
|            0|
+-------------+

Number of Payment Methods: 5
```

Part 4– Data Cleaning



```
Original Number of Records: 2964624
```

- **Remove duplicates**

```
Records After Removing Duplicates: 2964624
```

Explanation: dropDuplicates() is used to remove duplicate rows from the dataset.

- **Remove invalid trip distances**

```
Records After Removing Invalid Trip Distances: 2904253
```

Explanation: It can removes records where the trip distance is zero or negative.

- **Remove negative fares**

```
Records After Removing Negative Fares: 2870188
```

Explanation: It removes records with negative fare values.

- **Missing Values:**

```
-RECORD 0-----------------------
 VendorID              | 0
 tpep_pickup_datetime  | 0
 tpep_dropoff_datetime | 0
 passenger_count       | 115293
 trip_distance         | 0
 RatecodeID            | 115293
 store_and_fwd_flag    | 115293
 PULocationID          | 0
 DOLocationID          | 0
 payment_type          | 0
 fare_amount           | 0
 extra                 | 0
 mta_tax               | 0
 tip_amount            | 0
 tolls_amount          | 0
 improvement_surcharge | 0
 total_amount          | 0
 congestion_surcharge  | 115293
 Airport_fee           | 115293
```

Explanation: it shows the number of missing values in each column.

```
Records After Handling Missing Values: 2754895
Final Number of Records: 2754895
+--------+--------------------+---------------------+---------------+-------------+----------+-----
|VendorID|tpep_pickup_datetime|tpep_dropoff_datetime|passenger_count|trip_distance|RatecodeID|store
+--------+--------------------+---------------------+---------------+-------------+----------+-----
|2       |2024-01-01 00:47:21 |2024-01-01 00:51:58  |2              |1.13         |1         |N
|1       |2024-01-01 00:41:15 |2024-01-01 00:45:01  |3              |0.8          |1         |N
|2       |2024-01-01 00:42:51 |2024-01-01 01:16:30  |1              |4.43         |1         |N
|2       |2024-01-01 00:29:42 |2024-01-01 00:43:10  |1              |3.64         |1         |N
|2       |2024-01-01 00:40:27 |2024-01-01 01:04:32  |2              |5.9          |1         |N
|2       |2024-01-01 00:50:36 |2024-01-01 01:17:49  |2              |3.43         |1         |N
|2       |2024-01-01 00:27:14 |2024-01-01 00:34:39  |1              |0.95         |1         |N
|2       |2024-01-01 00:12:32 |2024-01-01 00:20:12  |2              |1.83         |1         |N
|2       |2024-01-01 00:55:46 |2024-01-01 01:08:28  |1              |2.25         |1         |N
|1       |2024-01-01 00:04:46 |2024-01-01 00:09:25  |2              |0.8          |1         |N
+--------+--------------------+---------------------+---------------+-------------+----------+-----
only showing top 10 rows
```

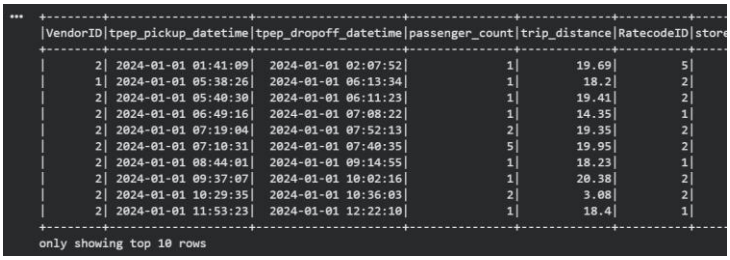
Explanation:

These steps ensured that the dataset was consistent, accurate, and suitable for further analysis. Missing values were identified by counting null values in every column and rows containing missing values were removed using dropna().

Part 5– Spark Transformations

filter()

Purpose: Find trips where the fare is greater than $50.

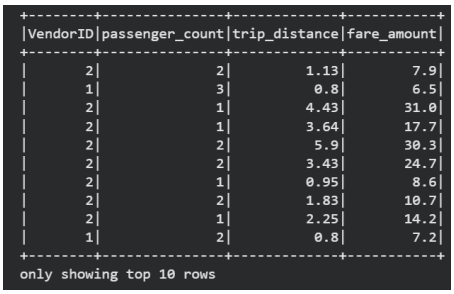


VendorID	tpep_pickup_datetime	tpep_dropoff_datetime	passenger_count	trip_distance	RatecodeID	store
2	2024-01-01 01:41:09	2024-01-01 02:07:52	1	19.69	5	
1	2024-01-01 05:38:26	2024-01-01 06:13:34	1	18.2	2	
2	2024-01-01 05:40:30	2024-01-01 06:11:23	1	19.41	2	
2	2024-01-01 06:49:16	2024-01-01 07:08:22	1	14.35	1	
2	2024-01-01 07:19:04	2024-01-01 07:52:13	2	19.35	2	
2	2024-01-01 07:10:31	2024-01-01 07:40:35	5	19.95	2	
2	2024-01-01 08:44:01	2024-01-01 09:14:55	1	18.23	1	
2	2024-01-01 09:37:07	2024-01-01 10:02:16	1	20.38	2	
2	2024-01-01 10:29:35	2024-01-01 10:36:03	2	3.08	2	
2	2024-01-01 11:53:23	2024-01-01 12:22:10	1	18.4	1	

only showing top 10 rows

select()

Purpose: Select only important columns.



VendorID	passenger_count	trip_distance	fare_amount
2	2	1.13	7.9
1	3	0.8	6.5
2	1	4.43	31.0
2	1	3.64	17.7
2	2	5.9	30.3
2	2	3.43	24.7
2	1	0.95	8.6
2	2	1.83	10.7
2	1	2.25	14.2
1	2	0.8	7.2

only showing top 10 rows

withColumn()

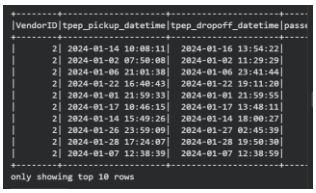
Purpose: Calculate fare per mile.

trip_distance	fare_amount	fare_per_mile
1.13	7.9	6.99
0.8	6.5	8.13
4.43	31.0	7.0
3.64	17.7	4.86
5.9	30.3	5.14
3.43	24.7	7.2
0.95	8.6	9.05
1.83	10.7	5.85
2.25	14.2	6.31
0.8	7.2	9.0

only showing top 10 rows

orderBy()

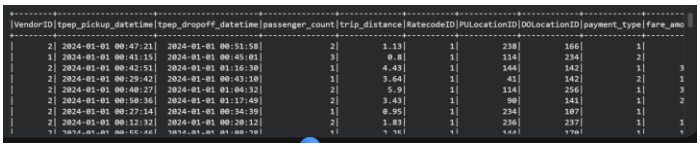
Purpose: Sort trips by highest fare.



```
+--------+--------------------+--------------------+-----
|VendorID|tpep_pickup_datetime|tpep_dropoff_datetime|passe
+--------+--------------------+--------------------+-----
|       2| 2024-01-14 10:08:11| 2024-01-16 13:54:22|
|       2| 2024-01-02 07:50:08| 2024-01-02 11:29:29|
|       2| 2024-01-06 21:01:38| 2024-01-06 23:41:44|
|       2| 2024-01-22 16:40:43| 2024-01-22 19:11:20|
|       2| 2024-01-01 21:59:33| 2024-01-01 21:59:55|
|       2| 2024-01-17 10:46:15| 2024-01-17 13:48:11|
|       2| 2024-01-14 15:49:26| 2024-01-14 18:00:27|
|       2| 2024-01-26 23:59:09| 2024-01-27 02:45:39|
|       2| 2024-01-28 17:24:07| 2024-01-28 19:50:30|
|       2| 2024-01-07 12:38:39| 2024-01-07 12:38:59|
+--------+--------------------+--------------------+-----
only showing top 10 rows
```

drop()

Purpose: Remove an unnecessary column.



```
+--------+--------------------+--------------------+---------------+-------------+----------+------------+------------+------------+---------+
|VendorID|tpep_pickup_datetime|tpep_dropoff_datetime|passenger_count|trip_distance|RatecodeID|PULocationID|DOLocationID|payment_type|fare_amo
+--------+--------------------+--------------------+---------------+-------------+----------+------------+------------+------------+---------+
|       2| 2024-01-01 00:47:21| 2024-01-01 00:51:58|              2|         1.13|         1|         238|         166|           1|
|       1| 2024-01-01 00:41:15| 2024-01-01 00:45:01|              3|          0.8|         1|         114|         234|           2|
|       2| 2024-01-01 00:42:51| 2024-01-01 01:16:30|              1|         4.43|         1|         144|         142|           1|        3
|       2| 2024-01-01 00:29:42| 2024-01-01 00:43:10|              1|         3.64|         1|          41|         142|           2|        1
|       2| 2024-01-01 00:40:27| 2024-01-01 01:04:32|              2|          5.9|         1|         114|         256|           1|        3
|       2| 2024-01-01 00:50:36| 2024-01-01 01:17:49|              2|         3.43|         1|          90|         141|           1|        2
|       2| 2024-01-01 00:27:14| 2024-01-01 00:34:39|              1|         0.95|         1|         234|         107|           1|
|       2| 2024-01-01 00:12:32| 2024-01-01 00:20:12|              2|         1.83|         1|         236|         237|           1|        1
```

distinct()

Purpose: Find unique payment methods.

```
+------------+
|payment_type|
+------------+
|           1|
|           3|
|           2|
|           4|
+------------+
```

groupBy()

Purpose: Calculate average fare for each payment method.

```
+------------+------------------+
|payment_type|      Average_Fare|
+------------+------------------+
|           1|18.376087258999238|
|           3|17.040781294018178|
|           2|18.613623047913922|
|           4| 19.66531404344601|
+------------+------------------+
```

join()

Purpose: Join the grouped data with payment

```
+------------+------------------+
|payment_type|      Average_Fare|
+------------+------------------+
|           1|18.376087258999238|
|           3|17.040781294018178|
|           2|18.613623047913922|
|           4| 19.66531404344601|
+------------+------------------+
```

alias()

Purpose: Rename columns using aliases.

```
+------+--------+----+
|Vendor|Distance|Fare|
+------+--------+----+
|     2|    1.13| 7.9|
|     1|     0.8| 6.5|
|     2|    4.43|31.0|
|     2|    3.64|17.7|
|     2|     5.9|30.3|
|     2|    3.43|24.7|
|     2|    0.95| 8.6|
|     2|    1.83|10.7|
|     2|    2.25|14.2|
|     1|     0.8| 7.2|
+------+--------+----+
only showing top 10 rows
```

repartition()

Purpose: Change the number of partitions for parallel processing.

```
repartitioned_taxi_df = clean_taxi_df.repartition(4)

print(
    "Number of Partitions:",
    repartitioned_taxi_df.rdd.getNumPartitions()
)

Number of Partitions: 4
```

Part 6– Spark SQL

Query 1: Top 10 longest trips

```
+-------------+-----------+---------------+
|trip_distance|fare_amount|passenger_count|
+-------------+-----------+---------------+
|     15400.32|       28.9|              1|
|     10879.28|       70.0|              1|
|      1715.22|       70.0|              1|
|         971.8|       21.5|              1|
|         964.6|       39.5|              1|
|         277.4|       33.8|              2|
|        246.22|        8.6|              1|
|        233.25|     1616.5|              1|
|        210.82|      500.0|              1|
|         210.2|      650.0|              1|
+-------------+-----------+---------------+
```

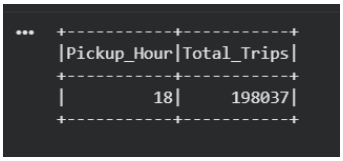
Query 2:Top Pickups Locations

```
+------------+-----------+
|PULocationID|Total_Trips|
+------------+-----------+
|         132|     138130|
|         237|     137093|
|         161|     136500|
|         236|     129604|
|         162|     102308|
|         186|     100737|
|         230|     100008|
|         142|      98906|
|         138|      87253|
|         239|      82391|
+------------+-----------+
```

Query 3: Average Fare by payment type

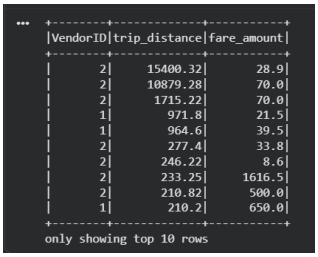
```
+------------+------------+
|payment_type|Average_Fare|
+------------+------------+
|           1|       18.38|
|           2|       18.61|
|           3|       17.04|
|           4|       19.67|
+------------+------------+
```

Query 4: Peak Pickup Hour



Pickup_Hour	Total_Trips
18	198037

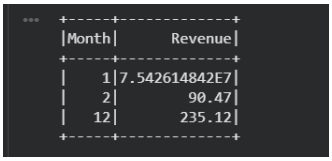
Query 5: Trips Over 20 Miles



VendorID	trip_distance	fare_amount
2	15400.32	28.9
2	10879.28	70.0
2	1715.22	70.0
1	971.8	21.5
1	964.6	39.5
2	277.4	33.8
2	246.22	8.6
2	233.25	1616.5
2	210.82	500.0
1	210.2	650.0

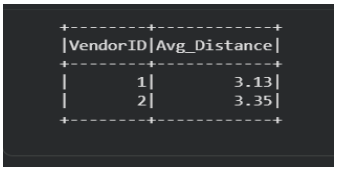
only showing top 10 rows

Query 6: Monthly Revenue



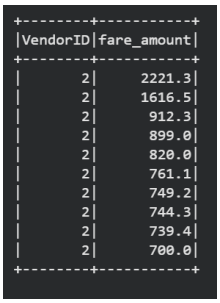
Month	Revenue
1	7.542614842E7
2	90.47
12	235.12

Query 7: Average Trip Distance By vendor



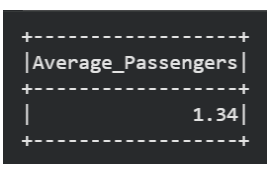
VendorID	Avg_Distance
1	3.13
2	3.35

Query 8: Top 10 Highest Fares



VendorID	fare_amount
2	2221.3
2	1616.5
2	912.3
2	899.0
2	820.0
2	761.1
2	749.2
2	744.3
2	739.4
2	700.0

QUERY 9 : Average Passenger Count



Average_Passengers
1.34

QUERY 10 — Trips by Payment Type

payment_type	Total_Trips
1	2298422
2	422945
4	22879
3	10649

Part 7– Window Functions

- row number()

Purpose: Assign a unique row number to each trip based on descending fare amount.

VendorID	fare_amount	row_number
2	2221.3	1
2	1616.5	2
2	912.3	3
2	899.0	4
2	820.0	5
2	761.1	6
2	749.2	7
2	744.3	8
2	739.4	9
2	700.0	10

only showing top 10 rows

- rank()

Purpose: Rank trips by fare amount. If two trips have the same fare, they receive the same rank, and the next rank is skipped.

VendorID	fare_amount	rank
2	2221.3	1
2	1616.5	2
2	912.3	3
2	899.0	4
2	820.0	5
2	761.1	6
2	749.2	7
2	744.3	8
2	739.4	9
2	700.0	10

only showing top 10 rows

• dense rank()

Purpose: Rank trips by fare amount without skipping rank numbers when there are ties.

VendorID	fare_amount	dense_rank
2	2221.3	1
2	1616.5	2
2	912.3	3
2	899.0	4
2	820.0	5
2	761.1	6
2	749.2	7
2	744.3	8
2	739.4	9
2	700.0	10

only showing top 10 rows

Part 8– Performance Optimization

Measure Execution Time Before Caching

```
+------------+-------+
|payment_type|  count|
+------------+-------+
|           1|2298422|
|           3|  10649|
|           2| 422945|
|           4|  22879|
+------------+-------+

Execution Time Before Caching: 21.31794548034668 seconds
```

Cache the Dataset

```
Dataset cached successfully!
```

Measure Execution Time After Caching

```
+------------+-------+
|payment_type|  count|
+------------+-------+
|           1|2298422|
|           3|  10649|
|           2| 422945|
|           4|  22879|
+------------+-------+

Execution Time After Caching: 4.6216442584991455 seconds
```

```
========== PERFORMANCE COMPARISON ==========
Before Caching: 21.31794548034668 seconds
After Caching: 4.6216442584991455 seconds
Caching improved execution time.
```

Demonstrate repartition()

```
Partitions Before: 200
Partitions After: 4
```

Show the Execution Plan with explain()

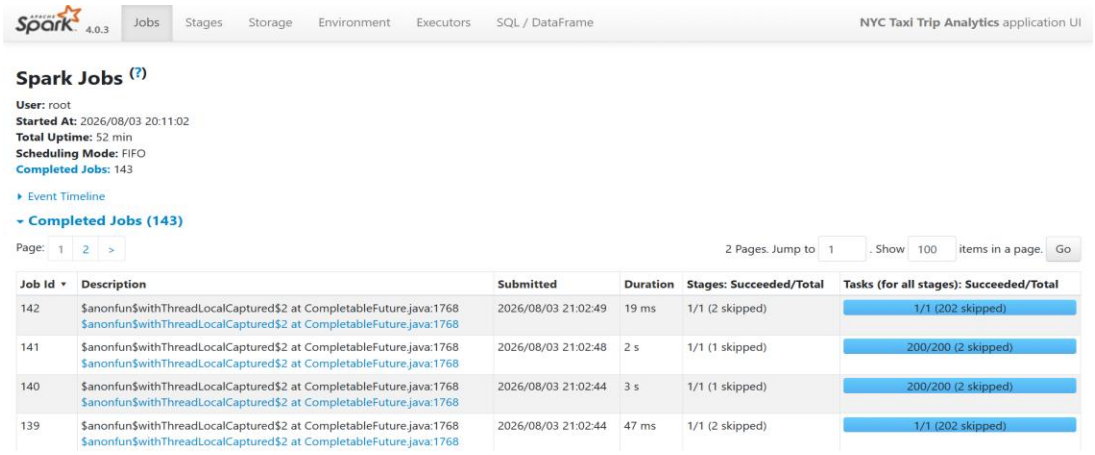
```
== Parsed Logical Plan ==
'Aggregate ['payment_type], ['payment_type, 'count(1) AS
+- Filter atleastnnonnulls(19, VendorID#0, tpep_pickup_d
   +- Filter (fare_amount#10 >= cast(0 as double))
      +- Filter (trip_distance#4 > cast(0 as double))
         +- Deduplicate [DOLocationID#8, improvement_sur
            +- Relation [VendorID#0,tpep_pickup_datetime

== Analyzed Logical Plan ==
payment_type: bigint, count: bigint
Aggregate [payment_type#9L], [payment_type#9L, count(1)
+- Filter atleastnnonnulls(19, VendorID#0, tpep_pickup_d
   +- Filter (fare_amount#10 >= cast(0 as double))
      +- Filter (trip_distance#4 > cast(0 as double))
         +- Deduplicate [DOLocationID#8, improvement_sur
            +- Relation [VendorID#0,tpep_pickup_datetime

== Optimized Logical Plan ==
Aggregate [payment_type#9L], [payment_type#9L, count(1)
+- Project [payment_type#9L]
   +- InMemoryRelation [VendorID#0, tpep_pickup_datetime
         +- AdaptiveSparkPlan isFinalPlan=true
            +- == Final Plan ==
```

Part 9– Spark UI Analysis

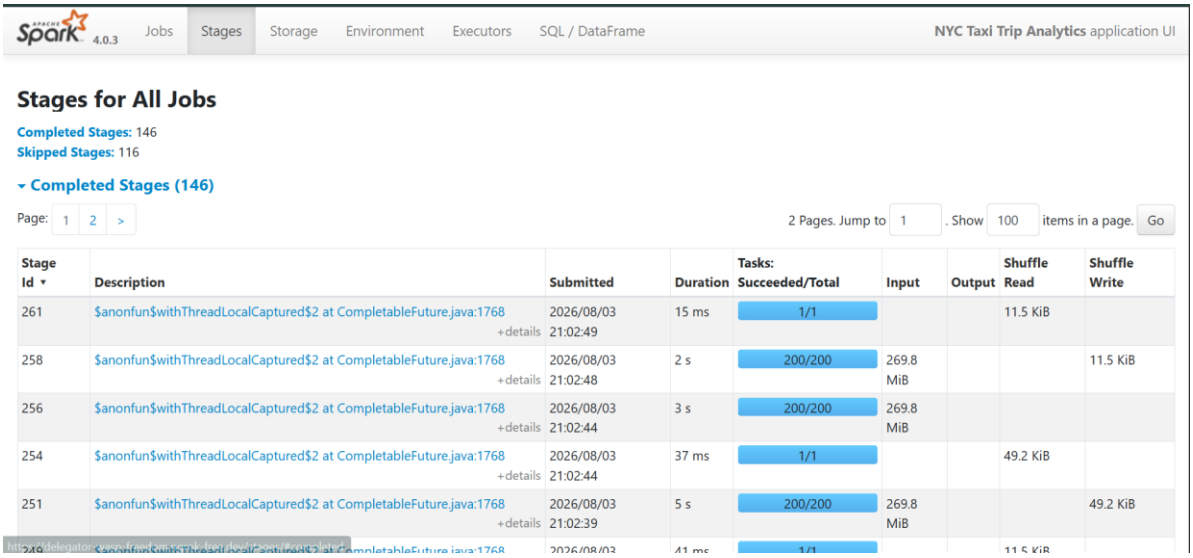
Jobs



Number of Jobs

Completed Jobs: 143

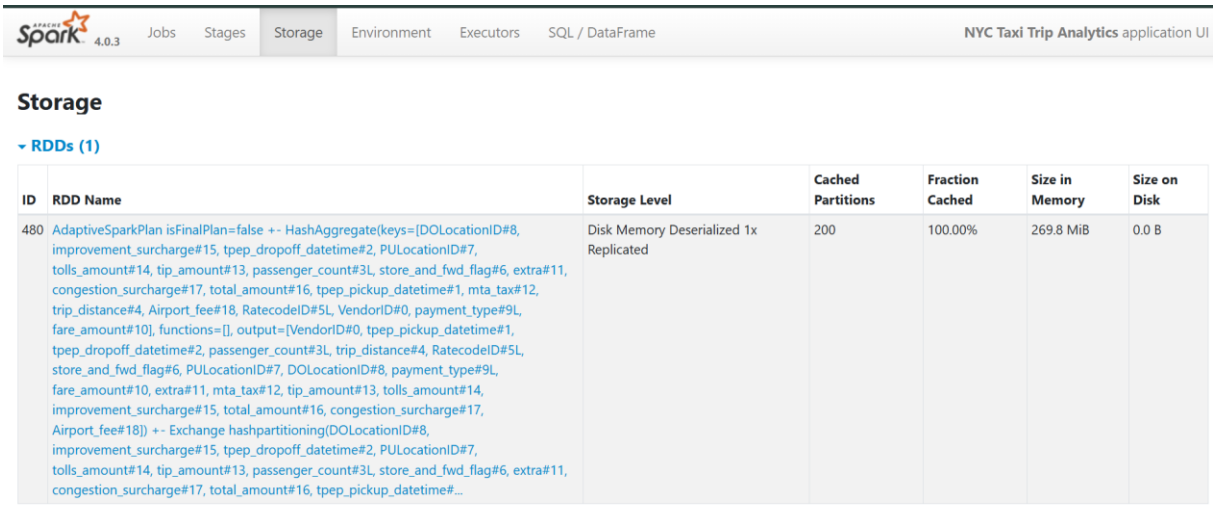
Stages



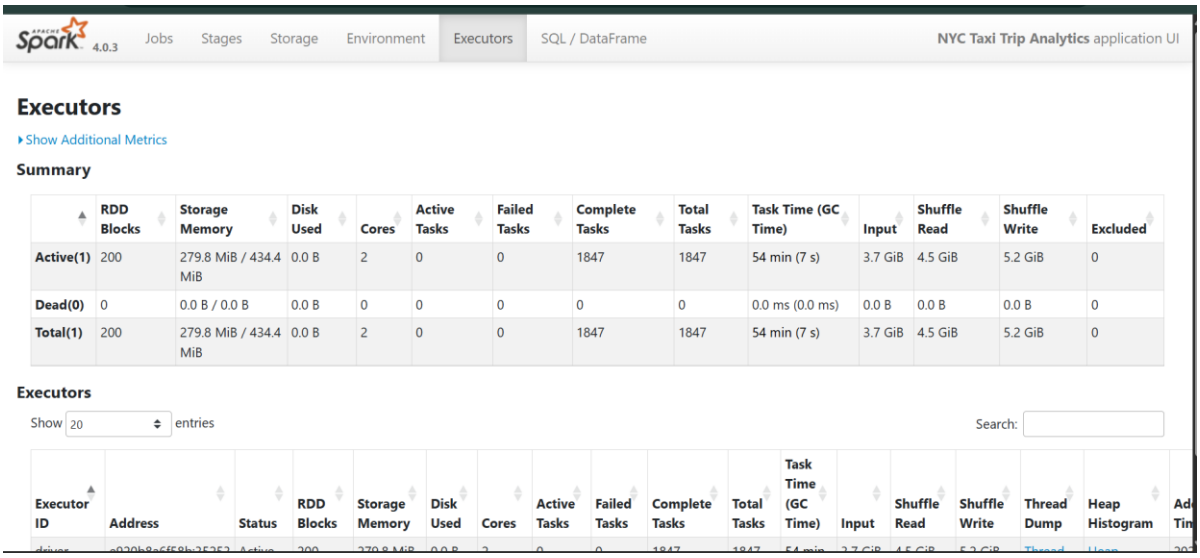
Number of Stages:

- **Completed Stages:** 146
- **Skipped Stages:** 116

Storage



Executor:



Which operation caused shuffle? Why?

Shuffle is the step in Spark where it shuffles the data between partitions in order to bring the related data into one place. For instance, when using `groupBy()` function on `payment_type`, it is possible to shuffle the data having the same payment type into one partition prior to the grouping by Spark. The similar situation occurs with the `orderBy()` function, since it may need shuffling the data across different partitions in order to have a globally sorted output.

Reflection

Through this assignment, I received practical insights on working with Apache Spark in the context of Google Colab. During this assignment, I gained skills in loading and exploring a big data set, doing data cleaning, applying various Spark transformations, and executing queries

using Spark SQL. Moreover, I got hands-on experience in working with window functions (`row_number()`, `rank()`, `dense_rank()`). Additionally, I learned how to use caching and repartitioning to optimize Spark performance. Working with the Spark User Interface allowed me to learn about Spark job, stage, and task organization. As Spark UI cannot be accessed from Google Colab directly, ngrok helped to connect and explore Spark UI.

One of the biggest difficulties that I encountered during this assignment was related to accessing and exploring Spark UI from Google Colab. The configuration of ngrok and connecting it to my Spark application took some extra time. Nevertheless, after establishing the connection, I managed to analyze Spark jobs and stages.

Another vital learning objective was understanding the distinction between Spark and Pandas. Pandas is usually helpful when you want to analyze small datasets, while Apache Spark is meant for processing huge datasets, and that is achieved through partitioning computation.

It was also good for me to gain some knowledge about lazy evaluation of Spark. Lazy evaluation means that functions like filter(), select(), and groupBy() will not run their computations but only build an execution plan and wait until there is an action, like count() or show(). Understanding the logic behind this approach made it easier for me to grasp Spark optimization.

In general, this task helped me gain more practical knowledge about Apache Spark and get to know how the capabilities of Spark can be used in large-scale data analysis.